ROUTEWISE

SMART TRAVEL PLANNING

PROJECT NOTEBOOK & TECHNICAL DESIGN

Version 0.3 | SQL / PostgreSQL Baseline | 7 November 2026

Prepared by
Naveed Mushtaq • Mohammad Shariq Khawaja • Nayeem

LIVING DOCUMENT

This notebook records the project as it is designed, built and tested. Planned work is not presented as completed work.

Document Control

Purpose: keep one accurate engineering record that can support development, team coordination, viva preparation and final academic submission.

Field	Value
Project	RouteWise — Smart Travel Planning
Version	0.3
Status	Planning baseline; implementation not yet verified
Database direction	PostgreSQL SQL database (confirmed direction)
Application stack	PERN: PostgreSQL, Express, React, Node
ORM/migrations	Prisma proposed; confirm during Phase 0
Target timeline	2–3 months
Target cash cost	₹0 target, subject to free-tier rules and provider terms
Team	Naveed Mushtaq; Mohammad Shariq Khawaja; Nayeem

Status labels

Status	Meaning
PROPOSED	Suggested; not yet approved or built
CONFIRMED	Decision approved by team
IN PROGRESS	Implementation started
IMPLEMENTED	Source code exists; tests may be pending
VERIFIED	Acceptance criteria passed and evidence recorded
BLOCKED	A blocker prevents progress
DEFERRED	Moved out of the current release

Mandatory update protocol

- Read the notebook and inspect actual files before every development phase.
- Work only on the phase the team requests.
- After each phase, update implementation log, exact file paths, component registry, ERD/migrations, API contracts, snippets, tests and issues.
- Never claim a feature is implemented or verified without source/test evidence.
- Keep passwords, tokens, production URLs and personal data out of documentation.
- The Markdown notebook in docs/ is the source of truth; DOCX/PDF/PPT are exports.

1. Project Overview

1.1 Summary

RouteWise is a web application intended to help travellers plan and manage a trip in one workspace. It combines trip details, map-based destination viewing, weather information, day-wise itineraries, budget planning, expense tracking and basic group expense sharing.

1.2 Problem statement

Travel information is often spread across separate map, weather, notes and expense tools. This makes it difficult to coordinate day plans, estimate a trip budget and reconcile shared expenses. RouteWise aims to organize these activities without becoming a booking or payment platform.

1.3 Objectives

- Centralize trip records, destinations and travel dates.
- Create day-wise plans with ordered activities and estimated costs.
- Show saved coordinates on a map and retrieve destination weather when a provider is available.
- Set a planned trip budget and track actual expenses.
- Record basic participant shares for group travel.
- Deliver a feasible MVP for a three-person team within approximately 2–3 months.

1.4 Scope

In scope — MVP	Out of scope — first release
Trip CRUD; basic account access (to confirm); members; itinerary days/items; map and weather; budget; expense records; basic splits; responsive UI.	Bookings; payment processing; bank/card storage; guaranteed live traffic; offline maps; AI trip generation; real-time collaboration/chat; native mobile apps.

MVP path: Create trip → add itinerary → set budget → record expense → review totals.

2. Technology Decisions

2.1 SQL decision

The user requested SQL instead of MongoDB. PostgreSQL is the recommended SQL database. RouteWise contains strongly related records—users, trips, members, itinerary days, activities, expenses and expense splits—so relational foreign keys, joins, constraints and transactions are a good fit.

2.2 Proposed stack

Layer	Technology	Responsibility	Status
Frontend	React + Vite + JavaScript/JSX	UI, routes, reusable components	PROPOSED
Backend	Node.js + Express.js	REST API, validation, authorization and business rules	PROPOSED
Database	PostgreSQL	Relational persistent data	CONFIRMED direction
ORM / migrations	Prisma ORM + Prisma Migrate	Database access and versioned schema migrations	PROPOSED
Maps	Leaflet + approved tile/geocoding provider	Map display and location search	PROPOSED
Weather	Open-Meteo candidate	Destination forecast lookup	PROPOSED; check current terms
Testing	Vitest/Jest, Supertest and browser tests	Unit, API, UI and regression verification	PROPOSED

The revised stack is called PERN: PostgreSQL, Express, React and Node. New documentation must not call it MERN.

2.3 Engineering rules

- PostgreSQL access happens only from the backend; never connect directly from browser code.
- Use UUID primary keys, foreign keys, UNIQUE/CHECK constraints, indexes and transactions.
- Use NUMERIC(12,2) and Prisma Decimal/equivalent for money; do not rely on JavaScript floating-point currency arithmetic.

- Use DATE for trip/day calendar dates, TIME for simple local activity times and TIMESTAMPTZ for UTC timestamps.
- Use versioned SQL migrations; do not change a deployed schema manually.
- Keep API keys and DATABASE_URL in server environment variables.

3. SRS — Software Requirements Specification

3.1 Functional requirements

ID	Requirement	Priority	Acceptance criteria	Status
FR-01	Account access	High	Access only owned/shared trips; login method finalized before auth build.	PROPOSED
FR-02	Create trip	High	Valid title, destination and date range create one trip.	PROPOSED
FR-03	Edit trip	High	Owner updates trip; invalid dates rejected.	PROPOSED
FR-04	List trips	High	Only owned/authorized trips appear.	PROPOSED
FR-05	Delete trip	Medium	Owner confirms deletion; child records follow explicit cascade rules.	PROPOSED
FR-06	Add trip members	Medium	Owner adds permitted members; duplicates rejected.	PROPOSED
FR-07	Destination coordinates	High	Latitude/longitude saved as a valid pair.	PROPOSED
FR-08	Map view	High	Valid saved coordinates appear on map.	PROPOSED
FR-09	Weather lookup	High	Forecast can be requested for a selected destination when supported.	PROPOSED
FR-10	Provider failure	High	Provider failure/quota shows safe message; stored trip still works.	PROPOSED
FR-11	Create itinerary day	High	Day belongs to trip and is within the approved date rule.	PROPOSED
FR-12	Manage activities	High	Authorized user adds/edits/removes/reorders activities.	PROPOSED
FR-13	Set budget	High	Owner sets non-negative budget and currency.	PROPOSED
FR-14	Record expense	High	Record title, amount, category, date and payer.	PROPOSED
FR-15	Calculate totals	High	Total spent and remaining budget derive consistently from saved expenses.	PROPOSED
FR-16	Split expense	Medium	Shares are non-negative and sum to expense total.	PROPOSED

FR-17	Edit/delete expense	High	Authorized update/delete recalculates totals correctly.	PROPOSED
FR-18	Currency	Medium	One currency per trip; INR default proposed; conversion out of scope.	PROPOSED
FR-19	UI feedback	High	Loading, empty, success, validation and error states are clear.	PROPOSED

3.2 Non-functional requirements

ID	Quality	Requirement
NFR-01	Usability	Common trip tasks understandable without training; clear labels and validation.
NFR-02	Responsive	Core screens usable at mobile, tablet and desktop widths.
NFR-03	Performance	Measure representative operations; do not invent targets or claims.
NFR-04	Reliability	Provider failures do not erase saved data or block core trip features.
NFR-05	Security	Server-side validation and authorization; no plaintext passwords.
NFR-06	Integrity	Foreign keys, constraints and transactions protect relational consistency.
NFR-07	Maintainability	Separate UI, API, service logic, data access and provider adapters.
NFR-08	Accessibility	Readable contrast, keyboard focus, labels and semantic controls.
NFR-09	Compatibility	Test current Chrome and one additional browser/device size.
NFR-10	Cost	₹0 target with free-service limits and terms documented.

3.3 Core acceptance flow

- Create a trip with valid dates.
- Add at least one itinerary day and activity.
- Set a planned budget.
- Record an expense of ₹500.
- Verify total increases by exactly ₹500 and remaining budget decreases by exactly ₹500.
- Reject invalid date range, negative amount and unauthorized private-trip access.
- Keep stored trip/expense functions usable when weather or map provider fails.

4. DRD — Design Requirements Document

4.1 Design principles

- Clean, conventional and readable travel-focused interface.
- Keep primary actions visible; avoid unnecessary dashboard decoration.
- Show planned, spent and remaining values distinctly.
- Maps/weather support trip management but must not block it.

- Use consistent typography, spacing, form fields and feedback.
- Confirm destructive actions; provide clear loading, empty, success and error states.

4.2 Screen inventory

Screen	Purpose	Proposed components	Status
/login, /register	Account access (subject to auth decision)	AuthForm, FormField, Alert	PROPOSED
/dashboard	Upcoming trips and summary	AppShell, PageHeader, TripCard, BudgetSummary, EmptyState	PROPOSED
/trips/new	Create trip	TripForm, DateRangeField, FormActions	PROPOSED
/trips/:tripId	Trip overview	TripHeader, TripOverview, WeatherCard, BudgetSummary	PROPOSED
/trips/:tripId/itinerary	Day-wise plan	ItineraryDayCard, ItineraryItem, ActivityForm, SortControls	PROPOSED
/trips/:tripId/map	Location map	MapView, MapMarker, PlaceDetailsPanel	PROPOSED
/trips/:tripId/expenses	Expenses and splits	ExpenseSummary, ExpenseList, ExpenseForm, SplitBreakdown	PROPOSED
/trips/:tripId/members	Trip participants	MemberList, MemberForm, RoleBadge	PROPOSED

4.3 Component registry

Component	Proposed path	Purpose / reason	Status
AppShell	client/src/components/layout/AppShell.jsx	Shared navigation/layout; prevents duplicated page frames.	PROPOSED
PageHeader	client/src/components/common/PageHeader.jsx	Consistent page heading and primary action.	PROPOSED
TripCard	client/src/features/trips/components/TripCard.jsx	Reusable summary card for dashboard and trip lists.	PROPOSED
TripForm	client/src/features/trips/components/TripForm.jsx	Shared create/edit form and validation.	PROPOSED
BudgetSummary	client/src/features/expenses/components/BudgetSummary.jsx	Consistent planned/spent/remaining display.	PROPOSED
ItineraryDayCard	client/src/features/itinerary/components/ItineraryDayCard.jsx	Groups activities for one trip date.	PROPOSED
ItineraryItem	client/src/features/itinerary/components/ItineraryItem.jsx	Displays one ordered activity/place.	PROPOSED
MapView	client/src/features/map/components/MapView.jsx	Isolates map rendering/provider integration.	PROPOSED
WeatherCard	client/src/features/weather/components/WeatherCard.jsx	Displays forecast and provider states.	PROPOSED
ExpenseForm	client/src/features/expenses/components/ExpenseForm.jsx	Expense input and validation.	PROPOSED
ExpenseList	client/src/features/expenses/components/ExpenseList.jsx	Reusable expense list with edit/delete.	PROPOSED
SplitBreakdown	client/src/features/expenses/components/SplitBreakdown.jsx	Shows participant shares and settlement state.	PROPOSED
EmptyState / LoadingState / ErrorState	client/src/components/feedback/	Consistent asynchronous and no-data feedback.	PROPOSED
ConfirmDialog	client/src/components/feedback/ConfirmDialog.jsx	Confirmation before destructive operations.	PROPOSED

For each real component, later record the exact path, props, API/data dependency, importing screen, reason for reuse, accessibility notes and linked tests.

5. ERD — Relational Database Design

5.1 Entity relationship overview

```
users 1:M trips
trips 1:M trip_members M:1 users
trips 1:M itinerary_days 1:M itinerary_items M:1 places (optional)
trips 1:M expenses 1:M expense_splits M:1 users
```

A trip has one owner via trips.owner_user_id, optional additional members, multiple itinerary days and expenses. Each expense records one payer and may contain multiple participant split rows.

5.2 Data dictionary

Table	Column	PostgreSQL type	Constraint / purpose
users	id	UUID	PK
users	full_name	VARCHAR(100)	NOT NULL
users	email	VARCHAR(255)	NOT NULL UNIQUE; lowercase normalized
users	password_hash	VARCHAR(255)	Only if password auth approved; never plaintext
trips	id	UUID	PK
trips	owner_user_id	UUID	FK → users.id
trips	title / destination_name	VARCHAR(150) / VARCHAR(180)	NOT NULL
trips	start_date / end_date	DATE	NOT NULL; end_date >= start_date
trips	currency	CHAR(3)	Default INR proposed
trips	planned_budget	NUMERIC(12,2)	CHECK >= 0
trip_members	trip_id / user_id	UUID / UUID	FKs; UNIQUE(trip_id,user_id)
trip_members	role / membership_status	VARCHAR(20) / VARCHAR(20)	Proposed member/viewer; pending/active
itinerary_days	trip_id / day_date	UUID / DATE	FK; UNIQUE(trip_id,day_date)
itinerary_days	day_order	INTEGER	CHECK >= 1; unique trip/day_order
places	id / name / address	UUID / VARCHAR(180) / TEXT	PK; reusable optional place
places	latitude / longitude	NUMERIC(9,6)	Both or neither; validate geographic range
itinerary_items	itinerary_day_id / place_id	UUID / UUID nullable	FKs; parent day and optional place
itinerary_items	title / description	VARCHAR(180) / TEXT	Title required
itinerary_items	start_time / end_time	TIME / TIME	Optional; end follows start
itinerary_items	estimated_cost / sort_order	NUMERIC(12,2) / INTEGER	cost >= 0; unique day/order
expenses	trip_id / paid_by_user_id	UUID / UUID	FKs; payer must belong to trip
expenses	amount	NUMERIC(12,2)	CHECK > 0
expenses	category / expense_date	VARCHAR(50) / DATE	Required classification and expense date
expense_splits	expense_id / user_id	UUID / UUID	FKs; UNIQUE(expense_id,user_id)
expense_splits	share_amount / is_settled	NUMERIC(12,2) / BOOLEAN	share >= 0; split sums to expense total

5.3 Integrity and transaction rules

- Use UUID primary keys, foreign keys and deliberate ON DELETE rules. Trip child rows may cascade from trip deletion; account deletion needs an explicit policy.
- Create/update an expense and its split rows in one SQL transaction.
- The sum of split rows must equal the parent expense amount. Enforce in service transaction and test it; a simple row-level CHECK cannot validate a child-row aggregate.
- Verify payer and every split participant is the trip owner or an active member.
- Use NUMERIC/Prisma Decimal for all currency calculations.
- Use parameterized queries; never concatenate user input into raw SQL.

5.4 Index plan

- Unique normalized users.email.
- trips(owner_user_id, updated_at DESC).
- Unique trip_members(trip_id, user_id).
- Unique itinerary_days(trip_id, day_date) and (trip_id, day_order).
- Unique itinerary_items(itinerary_day_id, sort_order).
- expenses(trip_id, expense_date DESC).
- Unique expense_splits(expense_id, user_id).

6. Architectural Design

6.1 High-level architecture

```
Browser
  ↓
React + Vite (pages, components, API client)
  ↓ HTTPS + JSON REST
Node.js + Express (routes → controllers → validation/auth → services)
  ↓
Prisma ORM + Prisma Migrate
  ↓
PostgreSQL (users, trips, members, itinerary, places, expenses, splits)

External adapters: weather / geocoding / map-provider services
```

The browser must never connect directly to PostgreSQL. Keep credentials and private provider keys on the server. Leaflet renders maps; the chosen tile/geocoding service has separate terms, attribution and usage limits.

6.2 Layer responsibilities

Layer	Responsibility	Proposed location
Frontend pages	Route-level screens and page composition	client/src/pages/ or feature route files
Reusable components	Layout, cards, fields, dialogs and feedback	client/src/components/
Feature components	Trip, itinerary, map, weather, expense UI	client/src/features/<feature>/
API client	Centralized HTTP and error normalization	client/src/services/api/
Routes/controllers	HTTP mapping and response formatting	server/src/routes/, controllers/
Services	Business rules, authorization, calculations and transactions	server/src/services/
Validators	Request schemas and field rules	server/src/validators/

Database	Prisma schema, migrations and client	server/prisma/, server/src/db/
External adapters	Weather/geocoding/routing integration	server/src/integrations/
Tests	Unit, API and integration coverage	server/tests/

6.3 Proposed API endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register, if approved
POST	/api/auth/login	Login, if approved
POST	/api/auth/logout	End session
GET	/api/auth/me	Current user
GET / POST	/api/trips	List / create trips
GET / PATCH / DELETE	/api/trips/:tripId	Read / update / delete trip
GET / POST	/api/trips/:tripId/members	List / add members
GET / POST	/api/trips/:tripId/itinerary-days	List / create days
POST	/api/itinerary-days/:dayId/items	Add activity
PATCH / DELETE	/api/itinerary-items/:itemId	Update / delete activity
GET / POST	/api/trips/:tripId/expenses	List / create expenses
PATCH / DELETE	/api/expenses/:expenseId	Update / delete expense
GET	/api/weather?lat=&lon=	Weather adapter
GET	/api/places/search?q=	Place search if approved

7. Testing Plan

Test ID	Requirement	Scenario	Expected result	Status
TC-01	FR-02	Create valid trip	One trip is persisted and returned.	NOT RUN
TC-02	FR-03	End date before start date	Validation error; no row saved.	NOT RUN
TC-03	FR-08	Trip with coordinates	Map marker appears or fallback shown.	NOT RUN
TC-04	FR-14	Add ₹500 expense	Exactly ₹500 is persisted once.	NOT RUN
TC-05	FR-16	Split ₹500 between members	Shares sum exactly ₹500.	NOT RUN
TC-06	NFR-05	Access another user private trip	Forbidden/not-found response per design.	NOT RUN
TC-07	FR-10	Weather provider unavailable	Core saved trip features remain usable.	NOT RUN

For each execution, record command, environment, date, expected and actual result, pass/fail and evidence path. Never claim a test passed from code inspection alone.

8. Code Snippet and Component Evidence Library

This section starts with placeholders. Replace them with short excerpts copied from actual source files after each feature is implemented.

ID	Snippet	Proposed path	Status
CS-01	React app shell and routes	client/src/app/	PLACEHOLDER
CS-02	TripForm with validation/API call	client/src/features/trips/components/TripForm.jsx	PLACEHOLDER

CS-03	Express trip route/service	server/src/routes/tripRoutes.js; server/src/services/tripService.js	PLACEHOLDER
CS-04	Prisma schema + SQL migration	server/prisma/schema.prisma; server/prisma/migrations/	PLACEHOLDER
CS-05	Expense transaction and split validation	server/src/services/expenseService.js	PLACEHOLDER
CS-06	Weather adapter and fallback	server/src/integrations/weather/	PLACEHOLDER
CS-07	Expense split unit/API test	server/tests/	PLACEHOLDER

Every entry must say what the code does, why it was used, which component/screen calls it, exact file path, linked requirement/test ID, verification result and commit/version where available. Never include secrets.

9. Phased Implementation Roadmap

Phase	Work package	Completion gate
0	Inspect workspace; confirm PostgreSQL/Prisma and authentication/member decisions; establish notebook.	Real starting state and blockers reported; no destructive changes.
1	React shell, navigation, responsive tokens, page placeholders and shared feedback components.	Routes render; responsive check; notebook registry updated.
2	Express API, PostgreSQL, Prisma schema/migration, health endpoint and trip CRUD.	Migration applies; trip CRUD tests pass against PostgreSQL.
3	Itinerary days/items, ordering and access checks.	Date/order tests pass.
4	Budget, expenses, Decimal-safe totals and transactional splits.	Money and split reconciliation tests pass.
5	Map/place and weather integrations with fallback states.	Provider policy/attribution verified; failure isolated.
6	Security, accessibility, responsive polish, regression and deployment prep.	Release checklist and evidence complete.

Build in parts. Do not start the next phase until the user requests it or approves the gate.

10. Decisions and Open Questions

ID	Topic	Current choice	Status
ADR-001	Database type	SQL instead of MongoDB	CONFIRMED by user
ADR-002	SQL engine	PostgreSQL	RECOMMENDED; confirm Phase 0
ADR-003	ORM/migrations	Prisma ORM + Prisma Migrate	PROPOSED; confirm Phase 0
ADR-004	Stack name	PERN	CONFIRMED consequence of SQL decision
ADR-005	Frontend language	JavaScript + JSX	PROPOSED default; preserve existing conventions
ADR-006	Authentication	Email/password or guest/demo	TO CONFIRM
ADR-007	Member permissions	Owner-only vs member roles	TO CONFIRM
ADR-008	Map/geocoding	Leaflet + selected provider	TO CONFIRM; review terms
ADR-009	Weather	Open-Meteo candidate	TO CONFIRM; review terms
ADR-010	Hosting	Select after verifying PostgreSQL support and free limits	TO CONFIRM

11. Phase-by-Phase Implementation Log

Current status: no application implementation has been verified. The project is at the planning/technical baseline stage.

Entry template — copy after each phase

Field	What to record
Date / Phase / Status	Date, P0/P1/..., IN PROGRESS / IMPLEMENTED / VERIFIED / BLOCKED
Objective and changes	What was built and why
Files	Exact files created and modified
Components	Name, path, props, used by which screen and why
Database/API	Migration names, constraints, endpoints and request/response
How it works	Short data flow and important logic
Tests	Command, expected, actual, pass/fail and evidence
Issues/decisions	Known bugs, blockers and pending approvals
Next approved step	Recommendation only; do not begin automatically

12. Final Submission Checklist

- Source code and repository structure match this notebook.
- SRS requirement statuses match real working screens/endpoints.
- DRD inventory and component registry contain actual paths and usage explanations.
- ERD matches actual Prisma schema and PostgreSQL migrations.
- Architecture matches the real application flow.
- Snippet library contains actual reviewed excerpts, not placeholders.
- Test register contains real commands/results/evidence.
- Known limitations and future scope are stated honestly.
- No secrets or private user data appear in the submitted files.
- Final DOCX/PDF/PPT exports are refreshed from the latest Markdown notebook.